

Introduction to Artificial Intelligence with Python

1. Artificial Intelligence

Search

Finding a solution to a problem, like a navigator app that finds the best route from your origin to the destination, or like playing a game and figuring out the next move.

Knowledge

Representing information and drawing inferences from it.

Uncertainty

Dealing with uncertain events using probability.

Learning

Improving performance based on access to data and experience. For example, your email is able to distinguish spam from non-spam mail based on past experience.

Neural Networks

A program structure inspired by the human brain that is able to perform tasks effectively.

Vision

Language

Processing natural language, which is produced and understood by humans.

Search

1

2

3

4

5

6

7

8

9

10

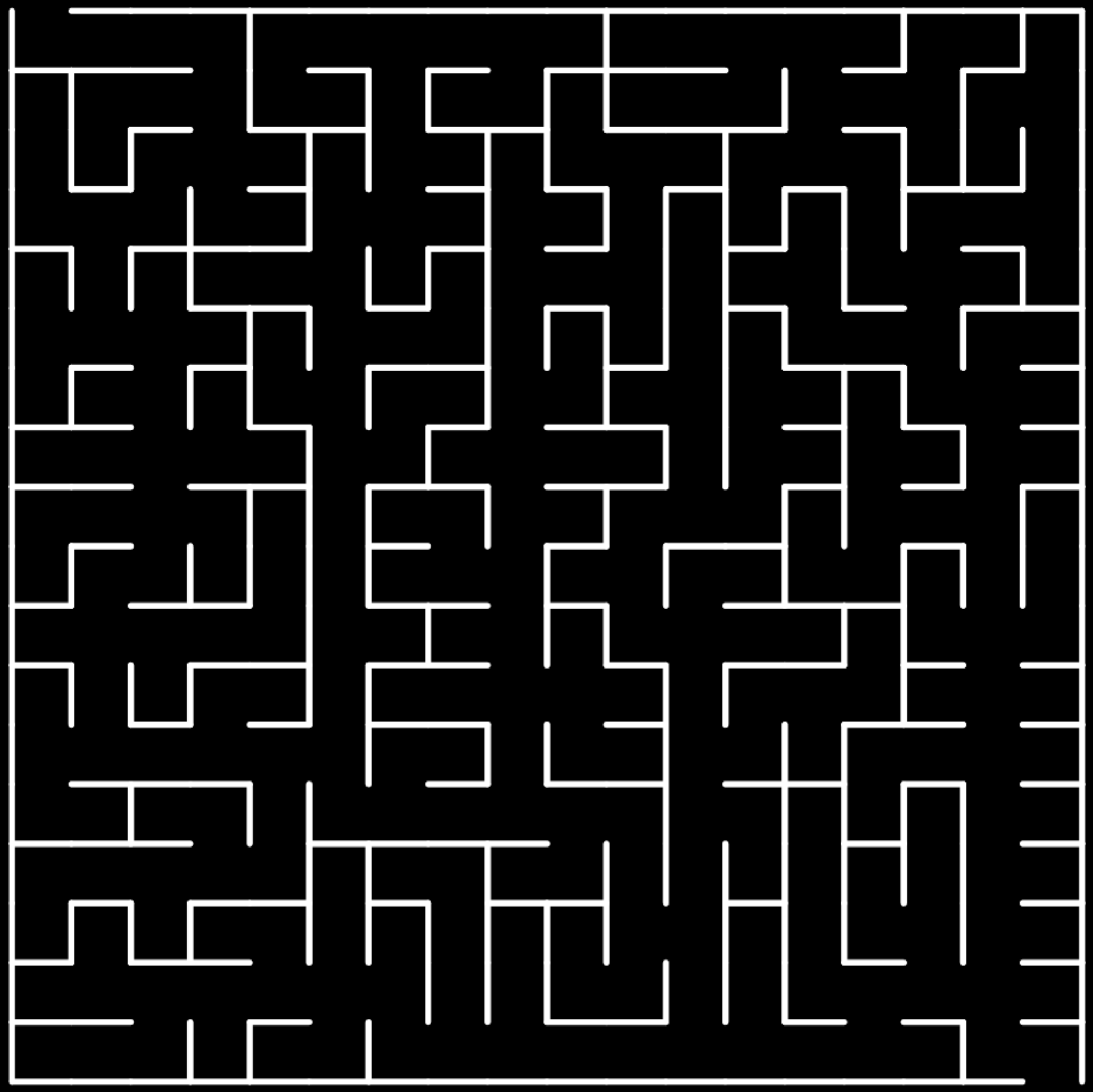
11

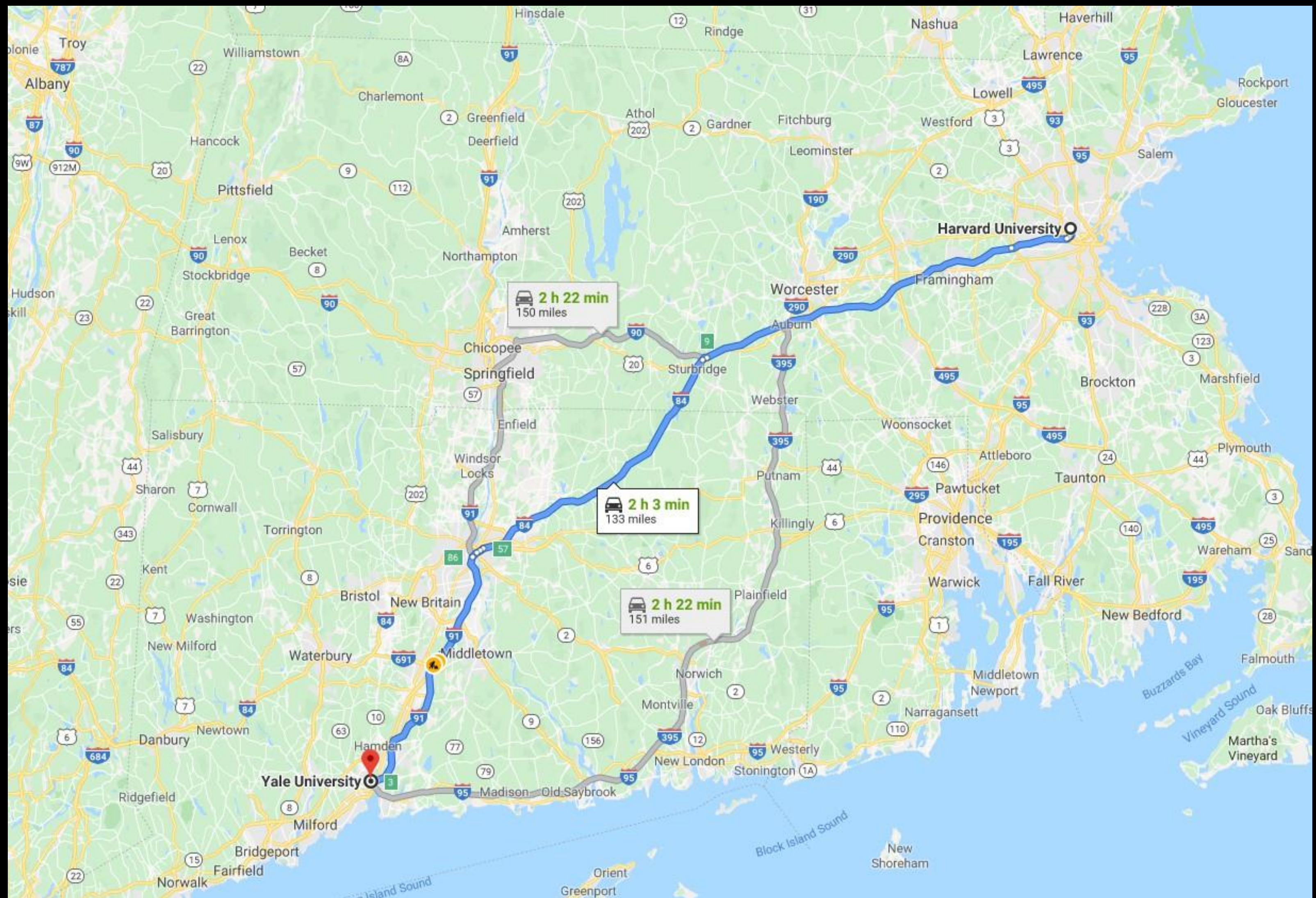
12

13

14

15





Search Problems

Agent (AI program)

- Look the problem and gives us the solution
- Entity that perceives its environment and acts upon that environment

state

a configuration of the agent and its environment

For example, in a 15 puzzle, a state is any one way that all the numbers are arranged on the board.

2	4	5	7
8	3	1	11
14	6		10
9	13	15	12

12	9	4	2
8	7	3	14
	1	6	11
5	13	10	15

15	4	10	3
13	1	11	12
9	5	14	7
6	8		2

initial state

the state in which the agent begins

initial state

The state from which the search algorithm starts. In a navigator app, that would be the current location

2	4	5	7
8	3	1	11
14	6		10
9	13	15	12

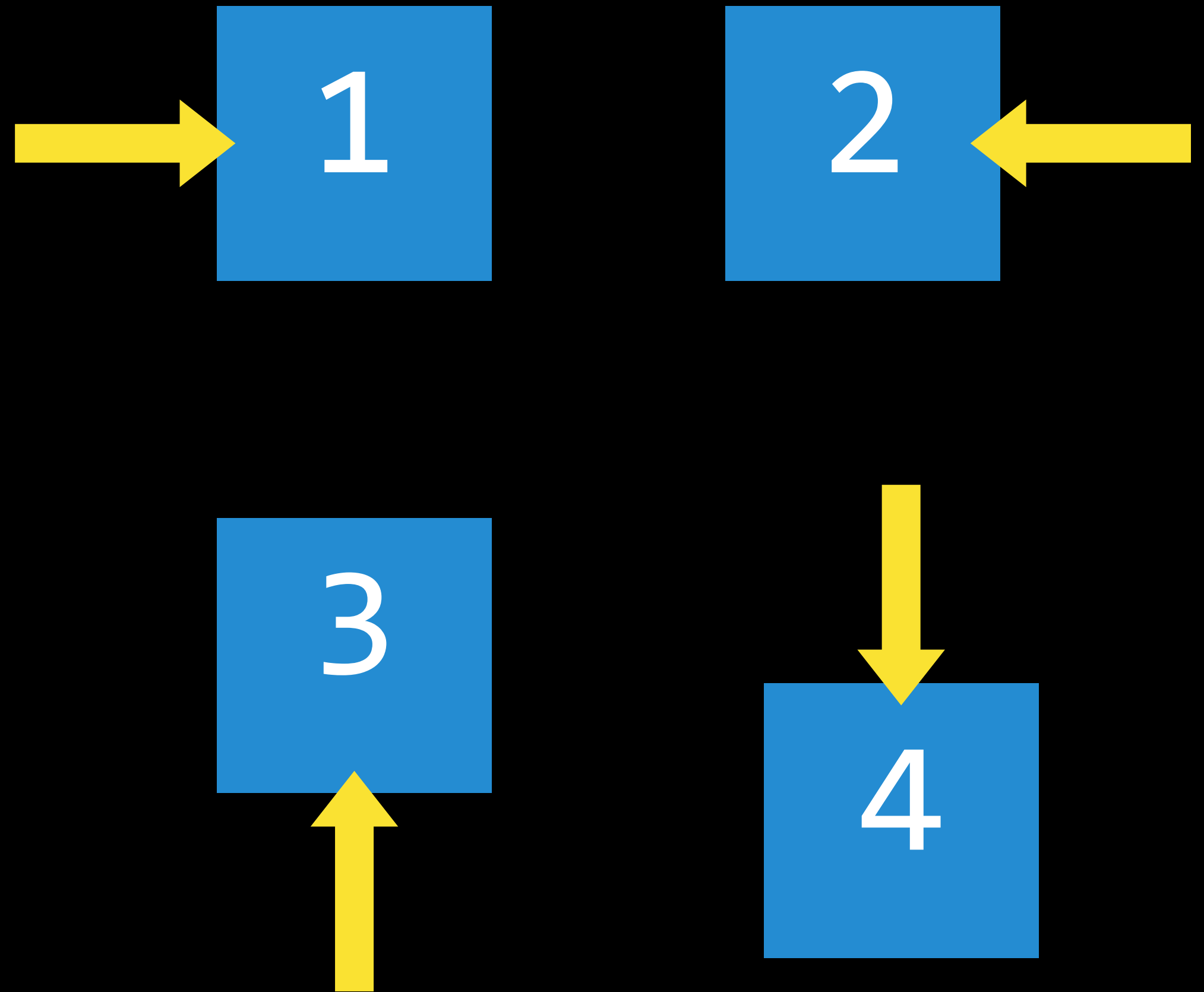
actions

choices that can be made in a state

actions

$ACTIONS(s)$ returns the set of actions that
can be executed in state s

actions



transition model

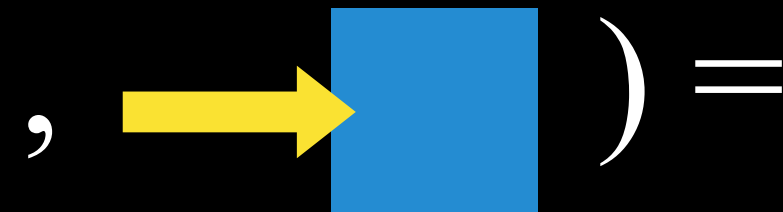
a description of what state results from
performing any applicable action in any
state

transition model

$\text{RESULT}(s, a)$ returns the state resulting from performing action a in state s

RESULT(

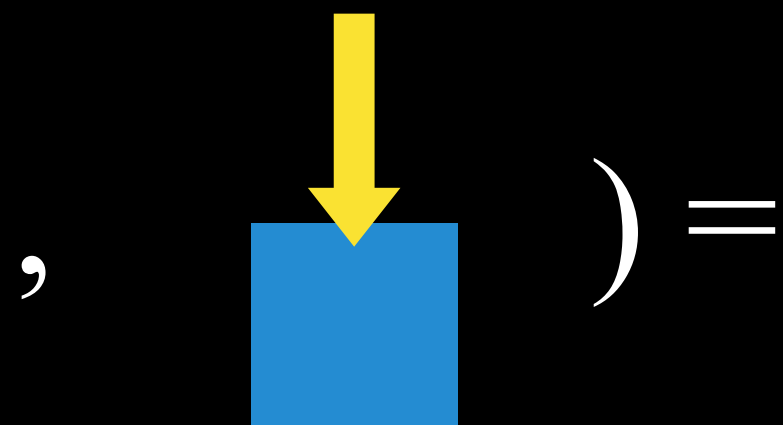
2	4	5	7
8	3	1	11
14	6	10	12
9	13	15	



2	4	5	7
8	3	1	11
14	6	10	12
9	13		15

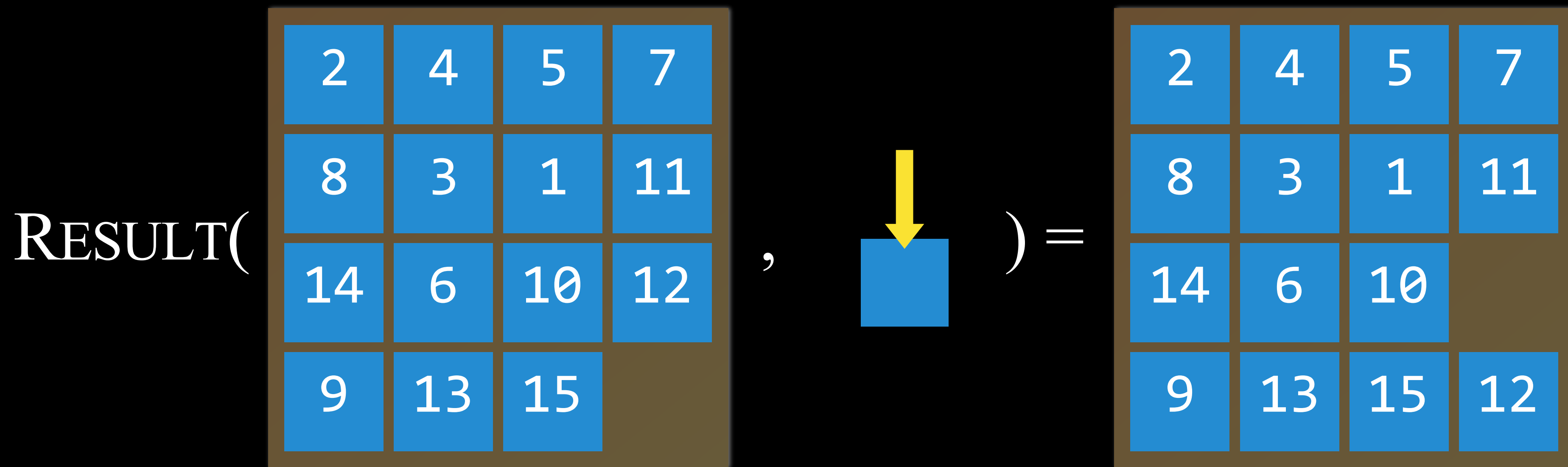
RESULT(

2	4	5	7
8	3	1	11
14	6	10	12
9	13	15	



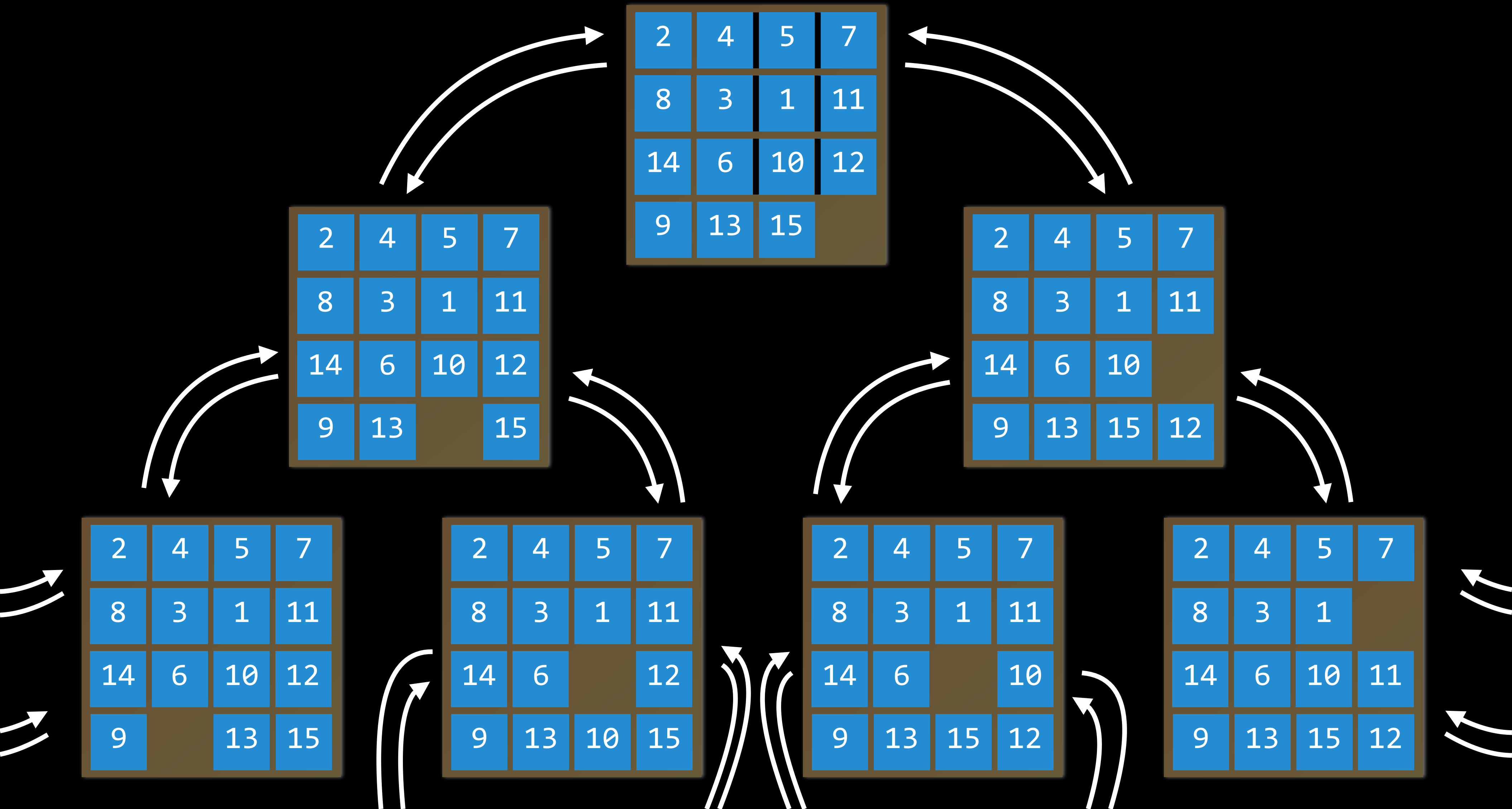
2	4	5	7
8	3	1	11
14	6	10	
9	13	15	12

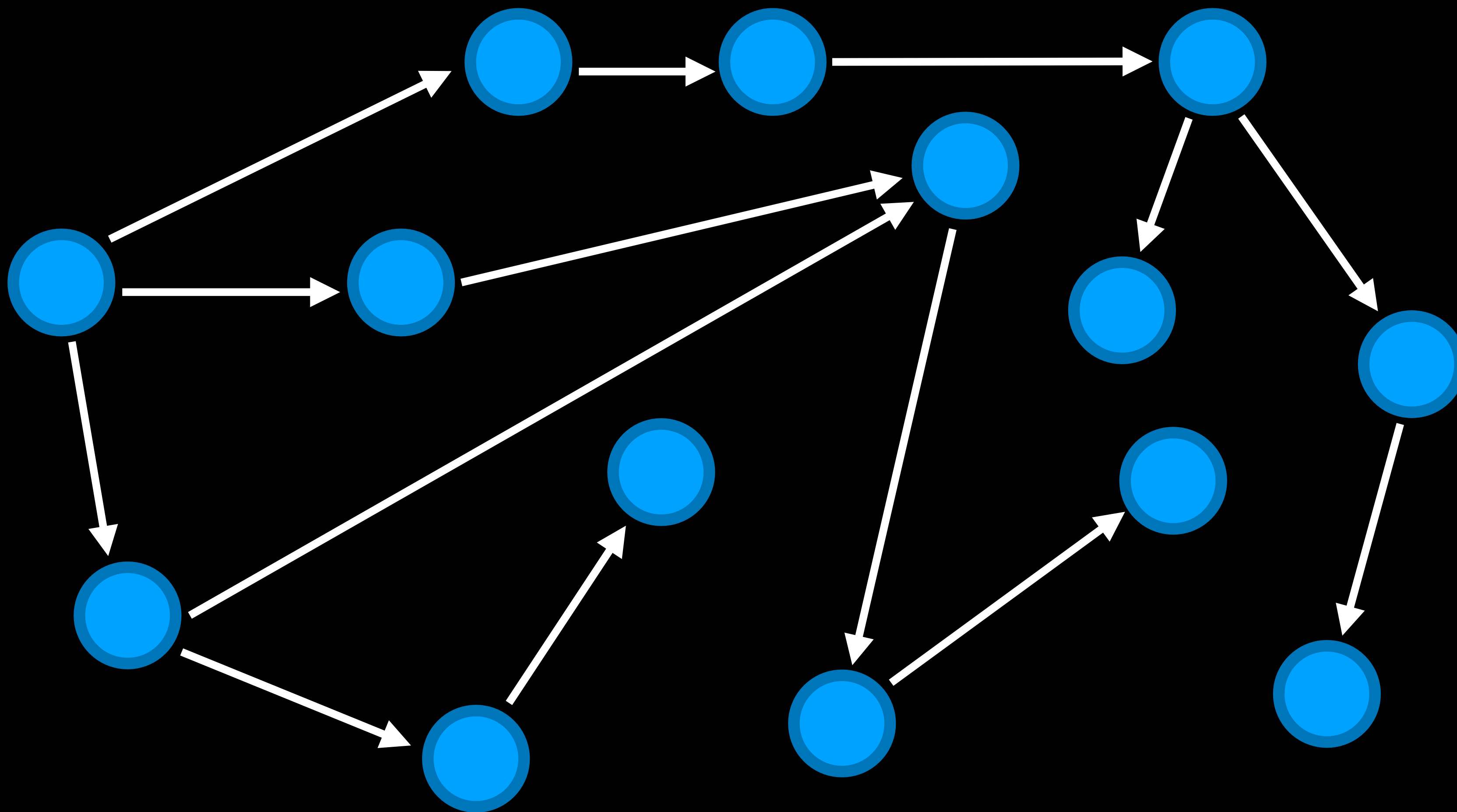
transition model



state space

the set of all states reachable from the initial state by any sequence of actions



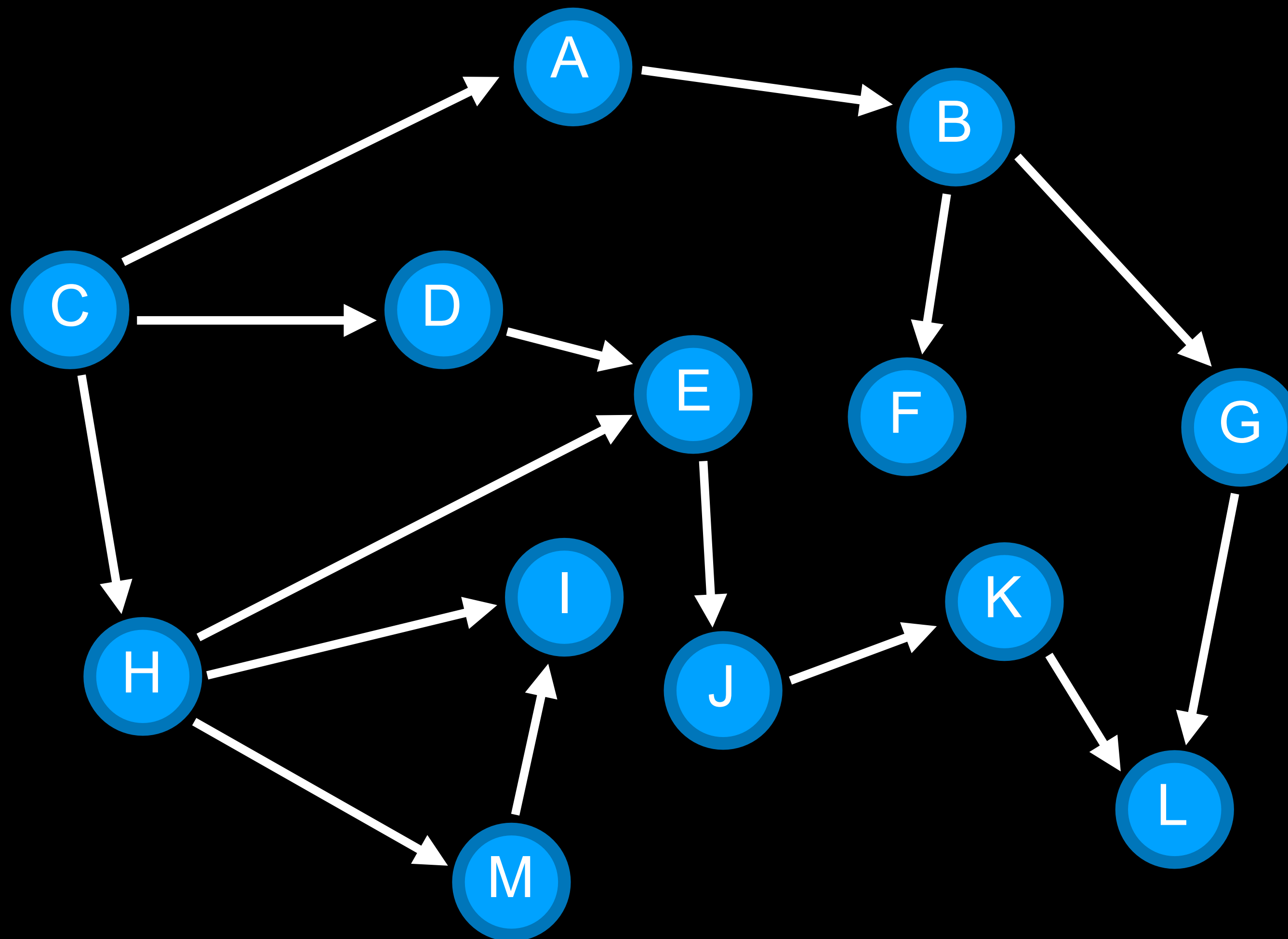


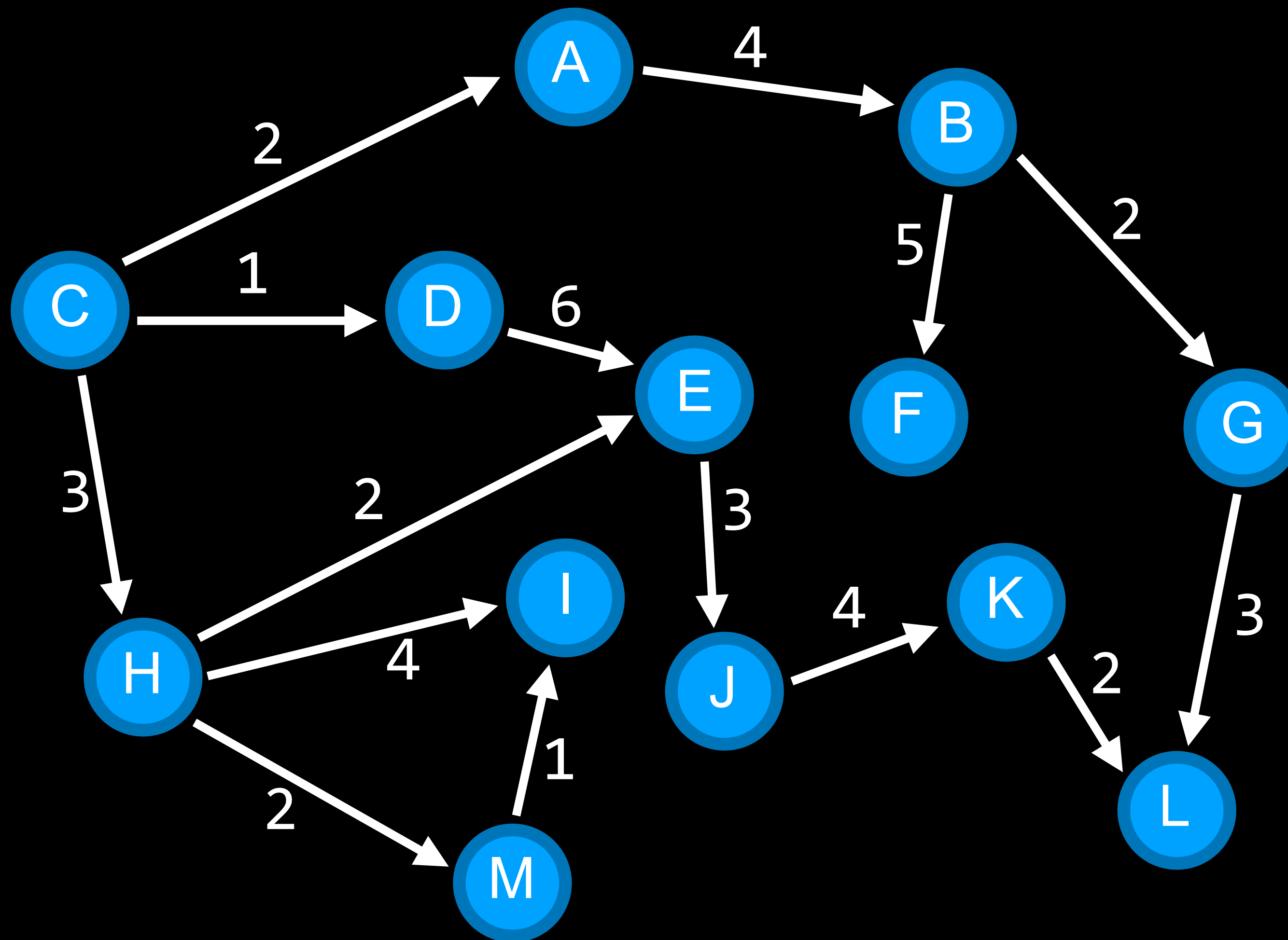
goal test

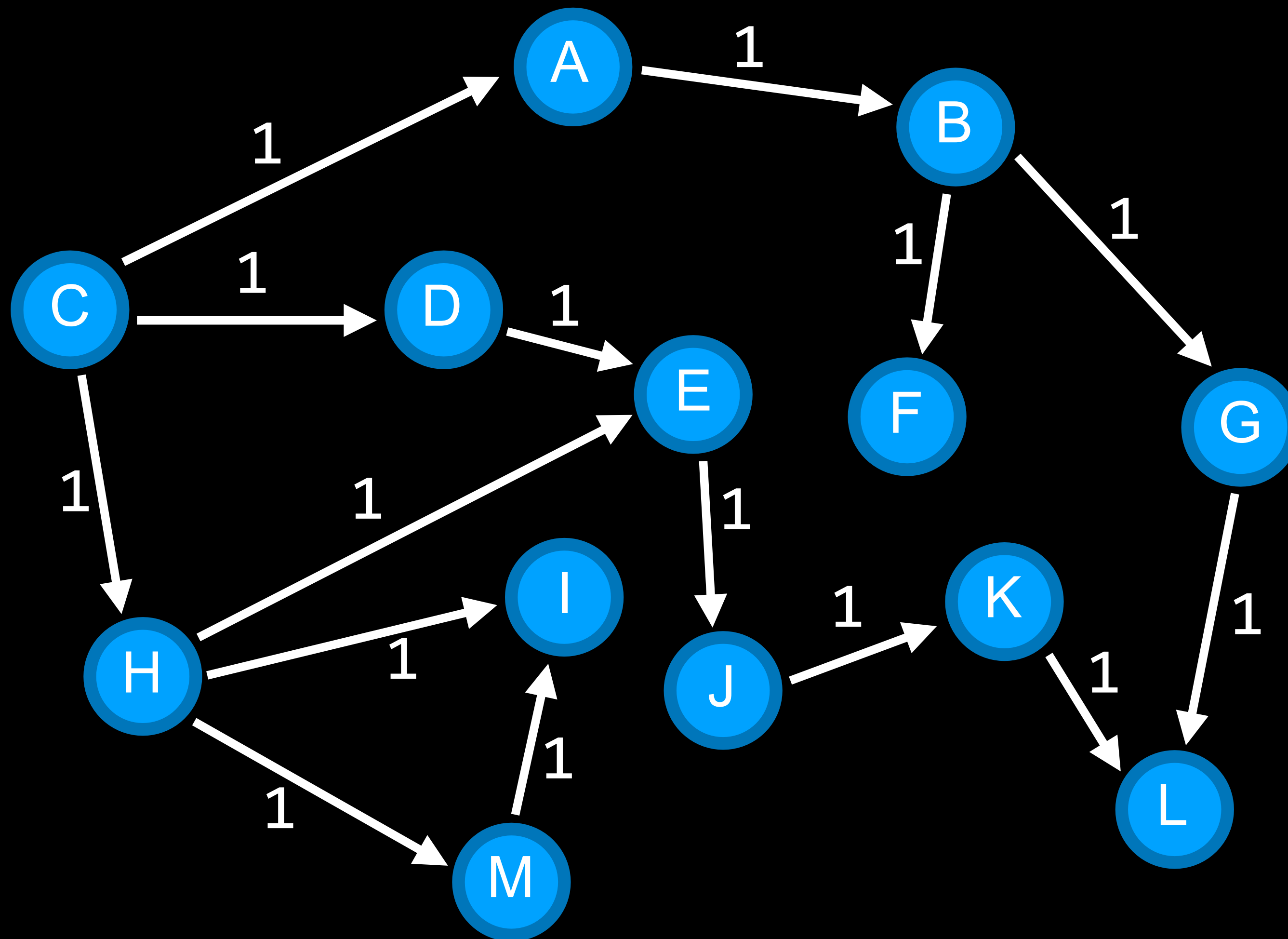
way to determine whether a given state
is a goal state

path cost

numerical cost associated with a given path







Search Problems

- initial state
- actions
- transition model
- goal test
- path cost function

solution

a sequence of actions that leads from the
initial state to a goal state

optimal solution

a solution that has the lowest path cost
among all solutions

node

a data structure that keeps track of

- a **state**
- a **parent** (node that generated this node)
- an **action** (action applied to parent to get node)
- a **path cost** (from initial state to node)

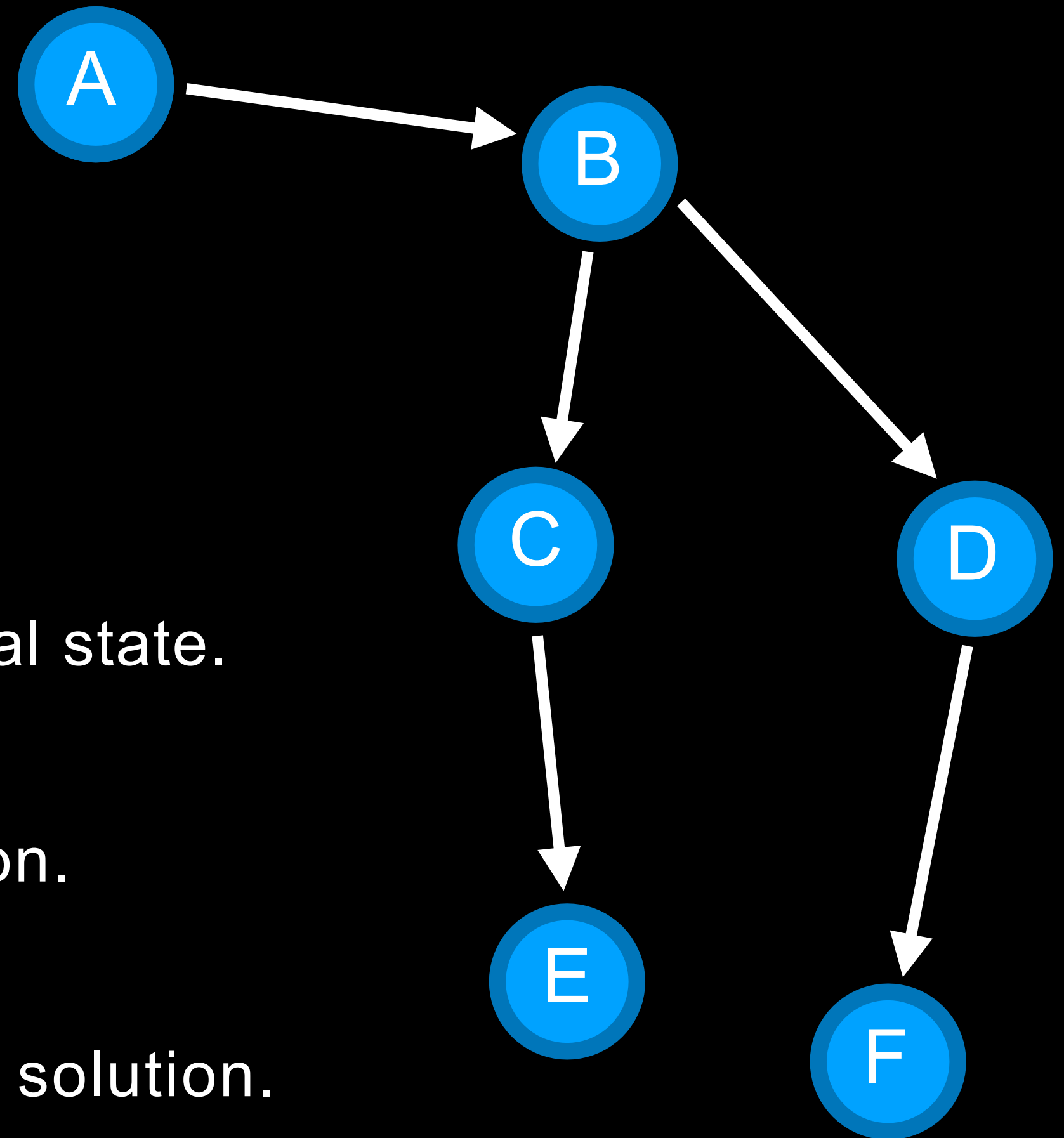
Approach

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **else, expand** node, add resulting nodes to the frontier.

Find a path from A to E.

Frontier

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

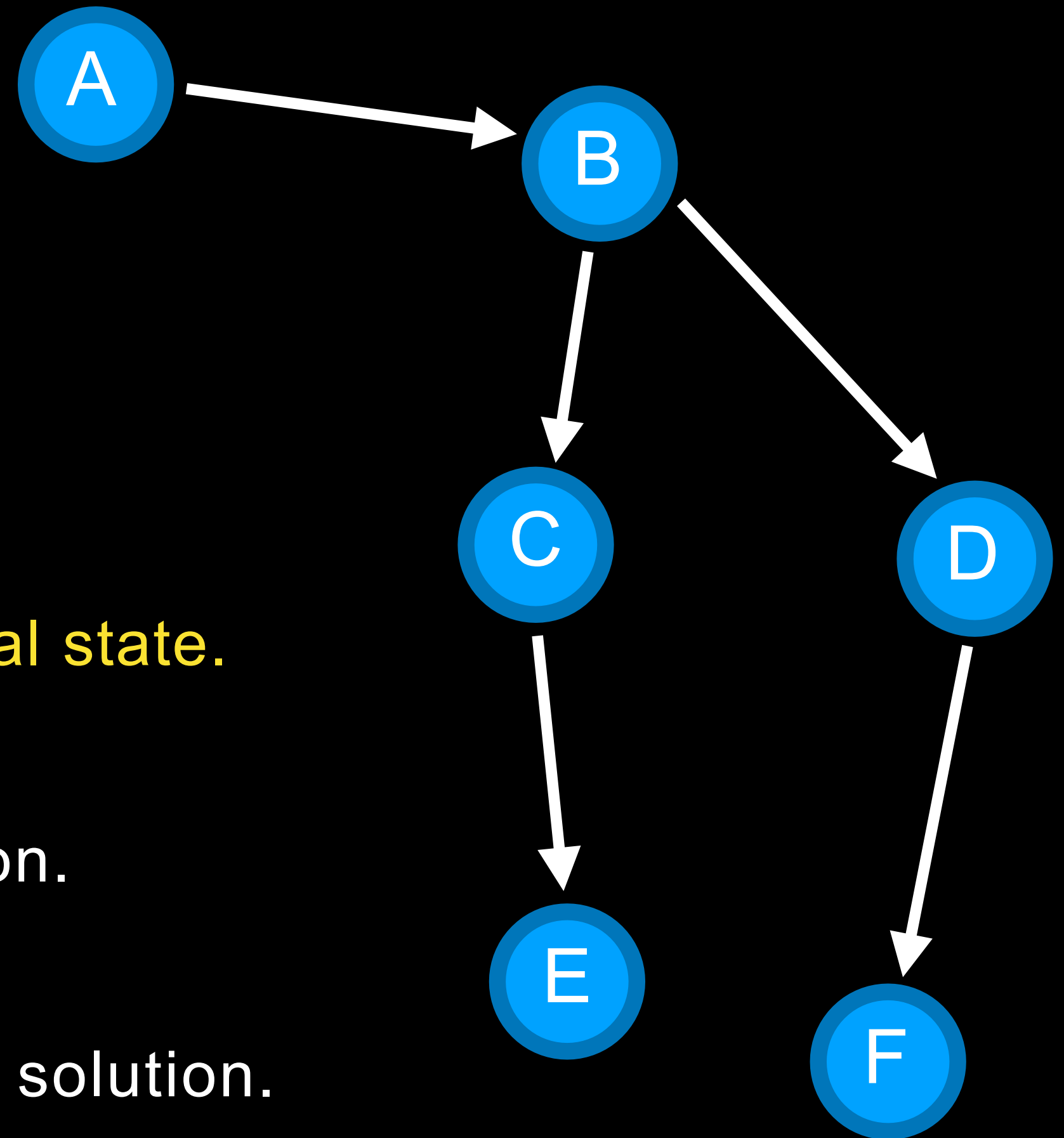


Find a path from A to E.

Frontier



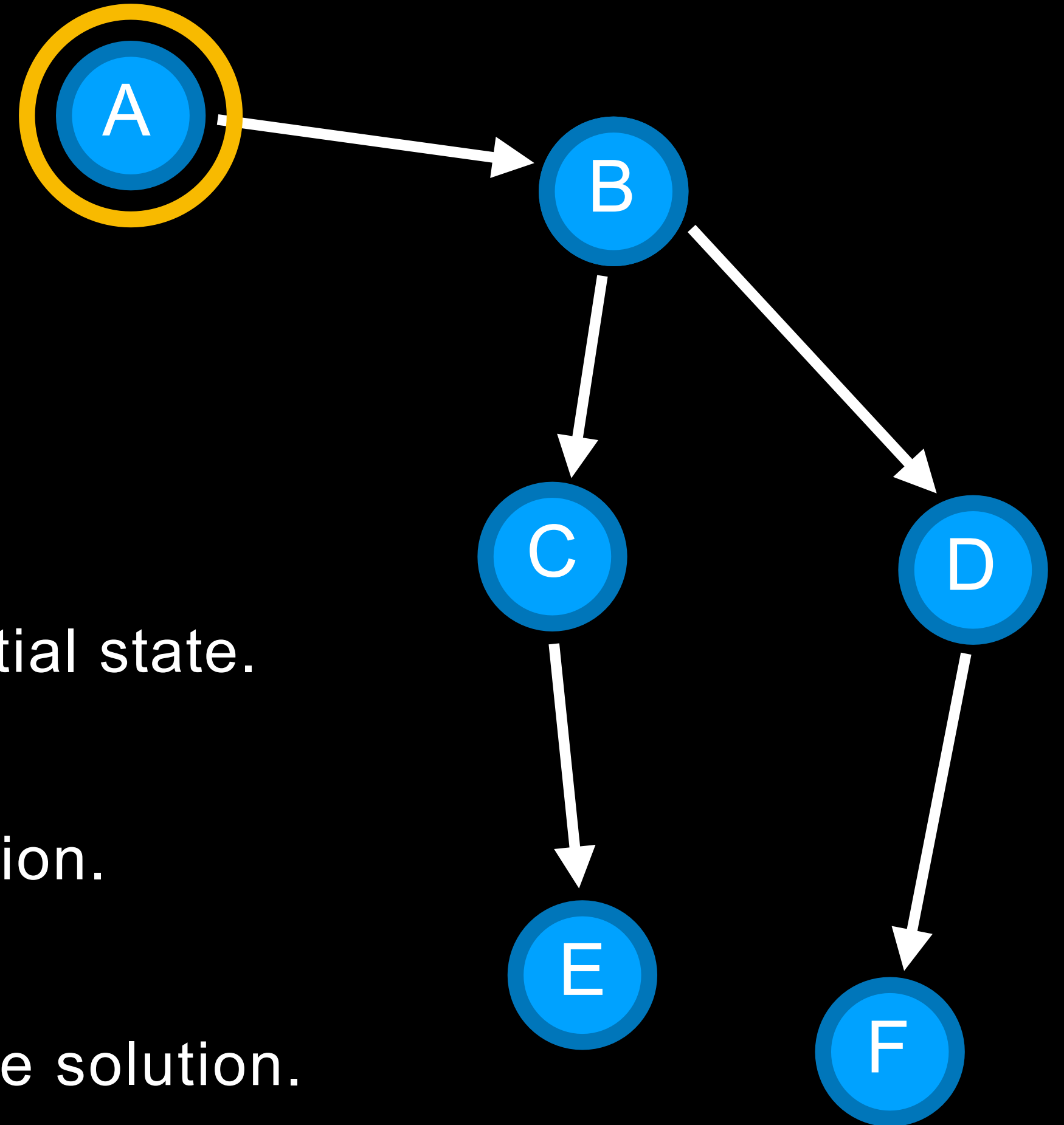
- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



Find a path from A to E.

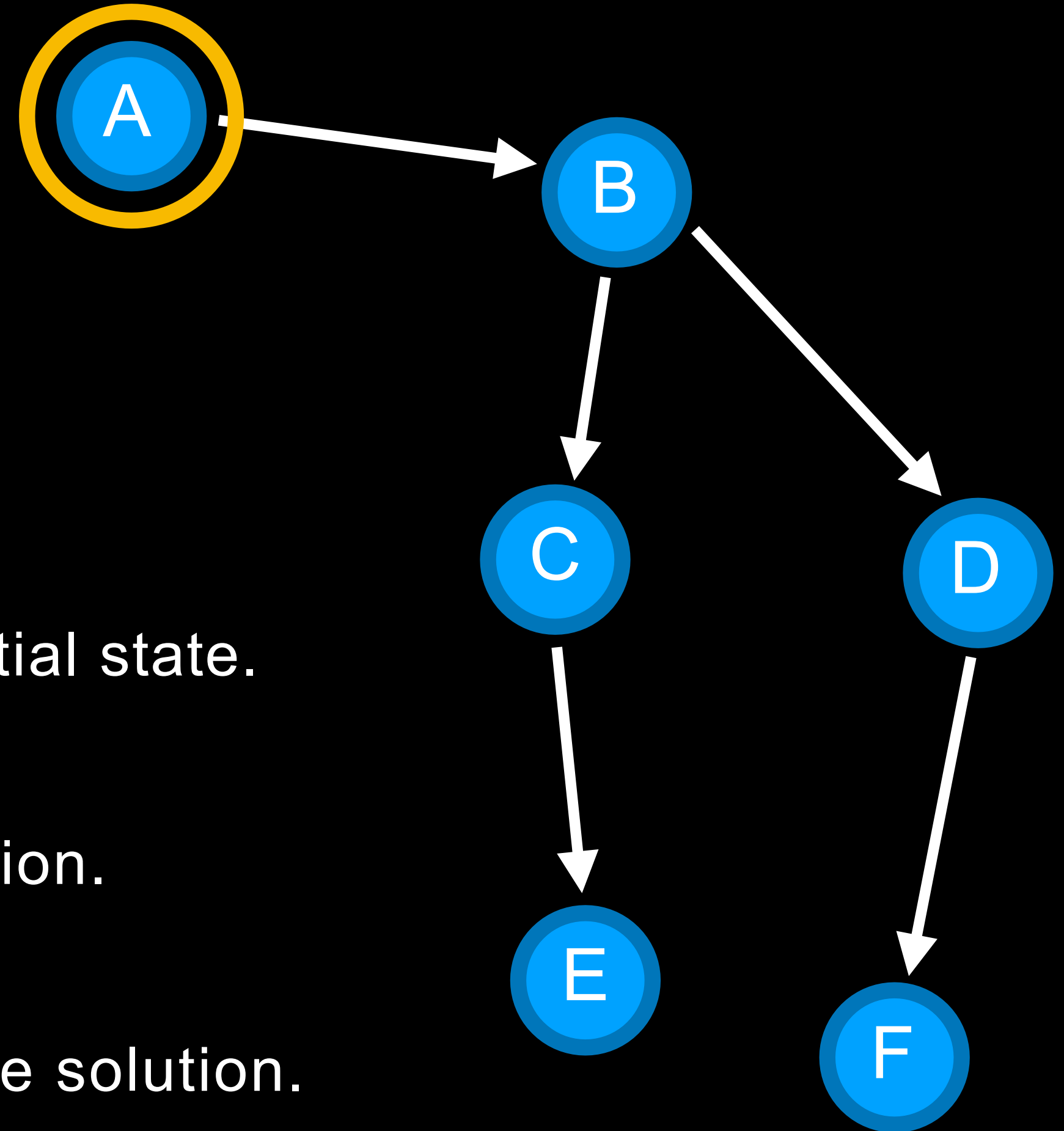
Frontier

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



Find a path from A to E.

Frontier

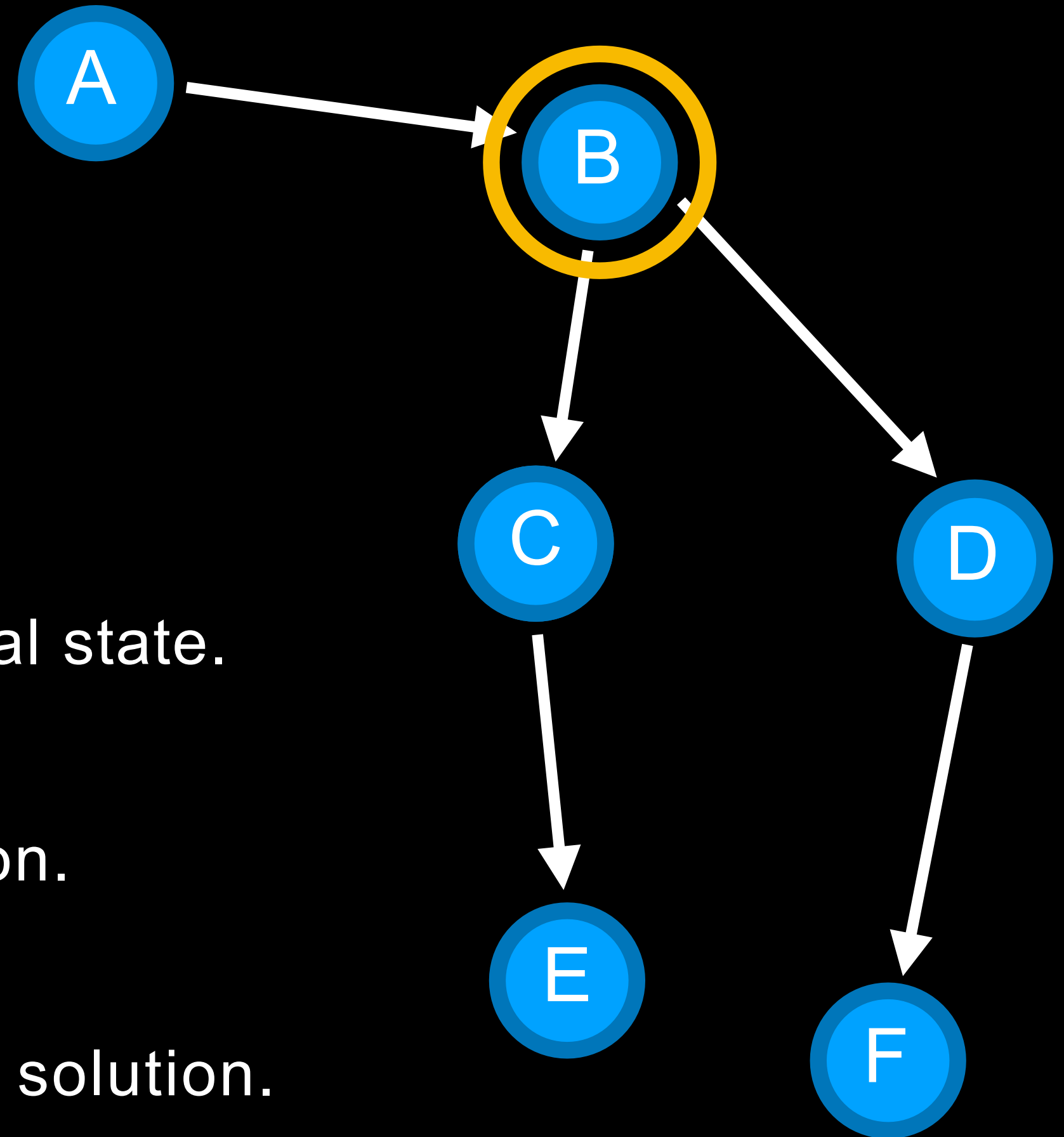


- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

Find a path from A to E.

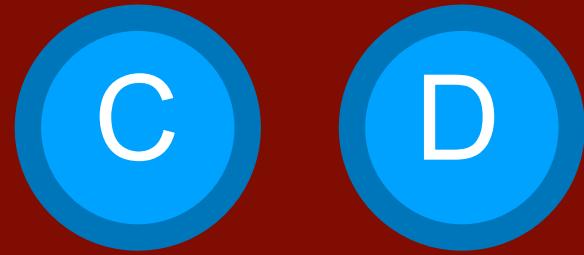
Frontier

- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

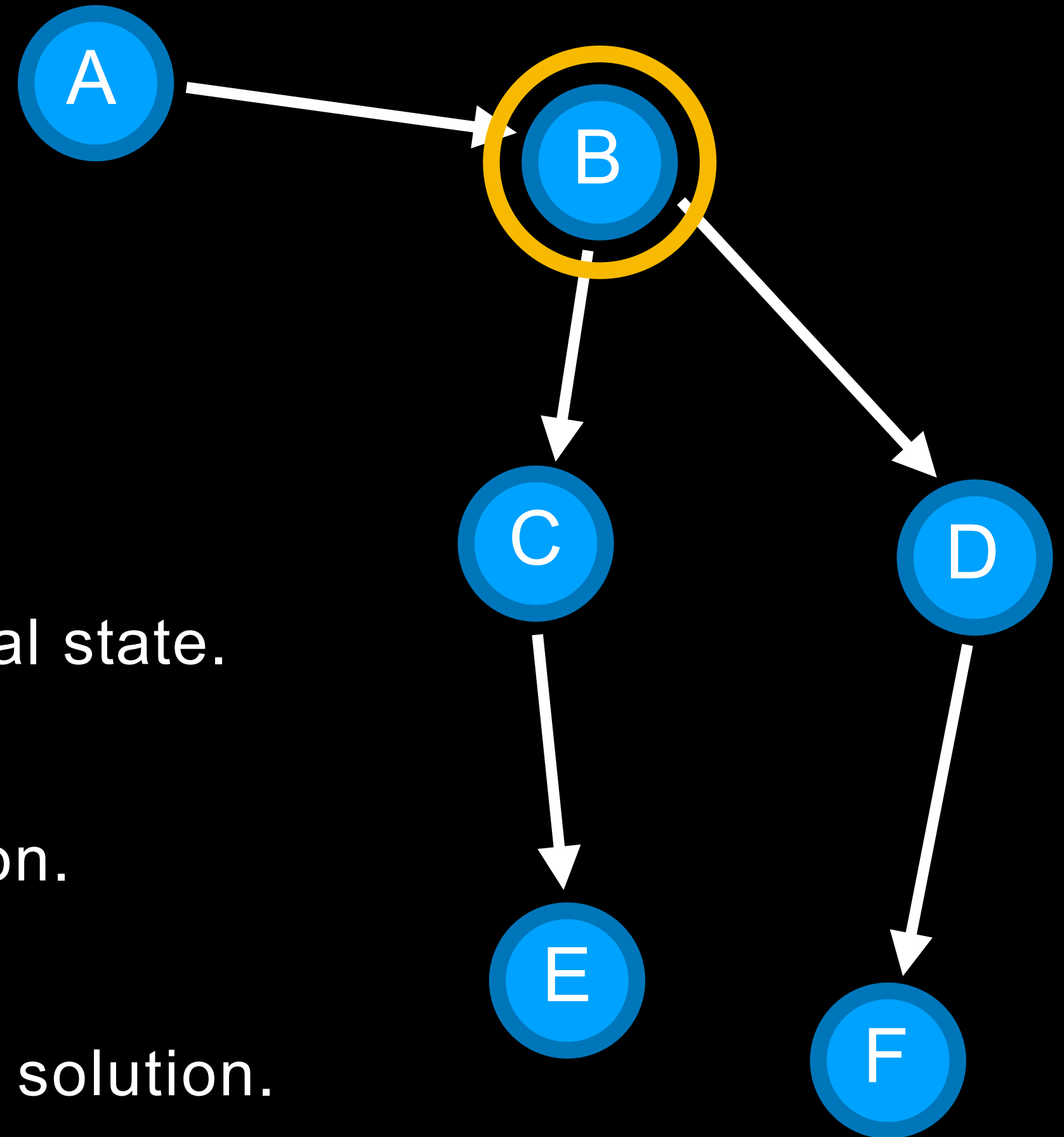


Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

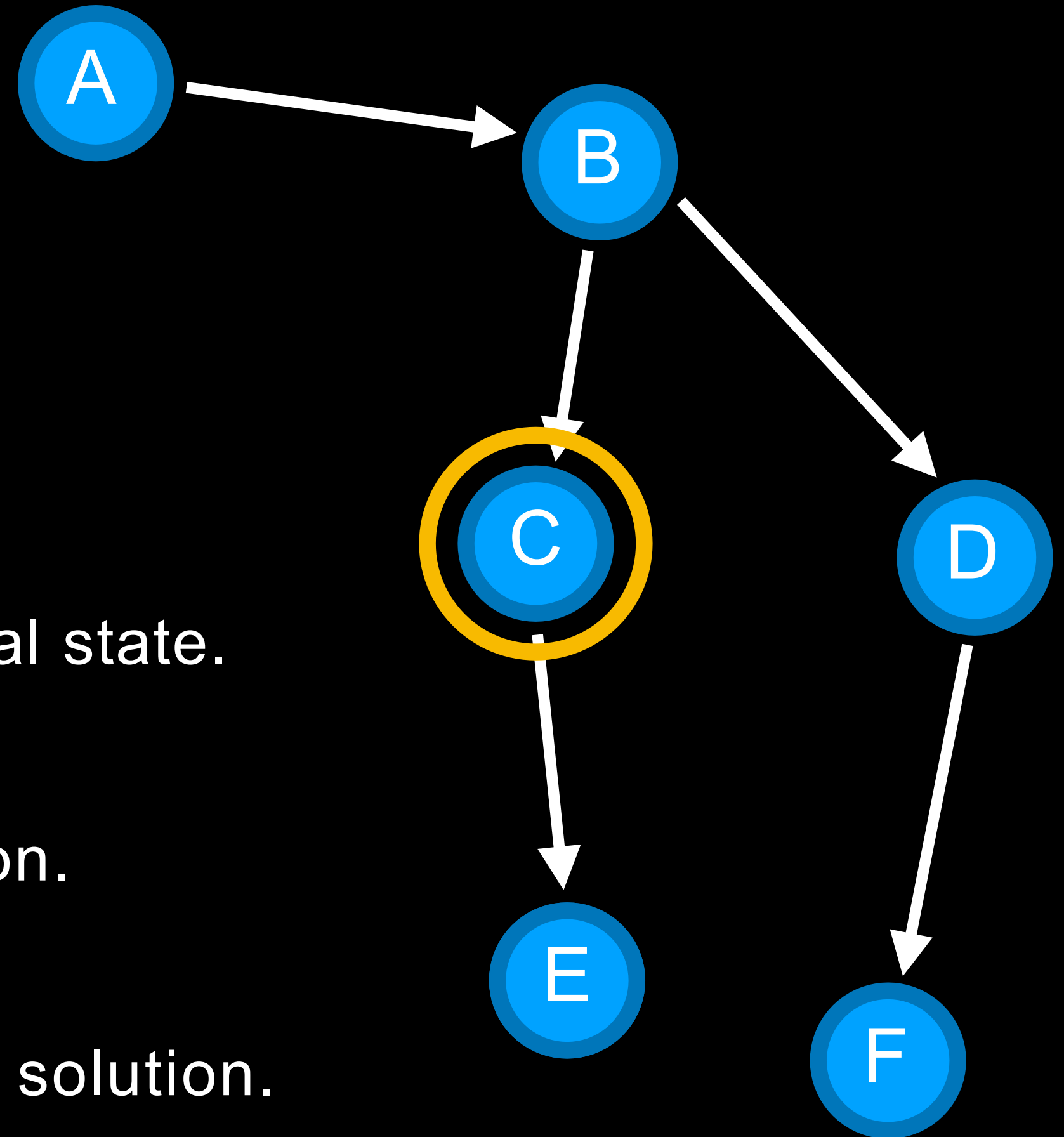


Find a path from A to E.

Frontier

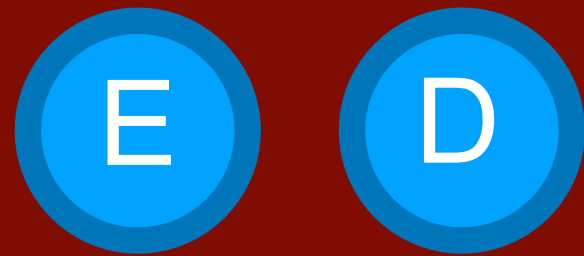


- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

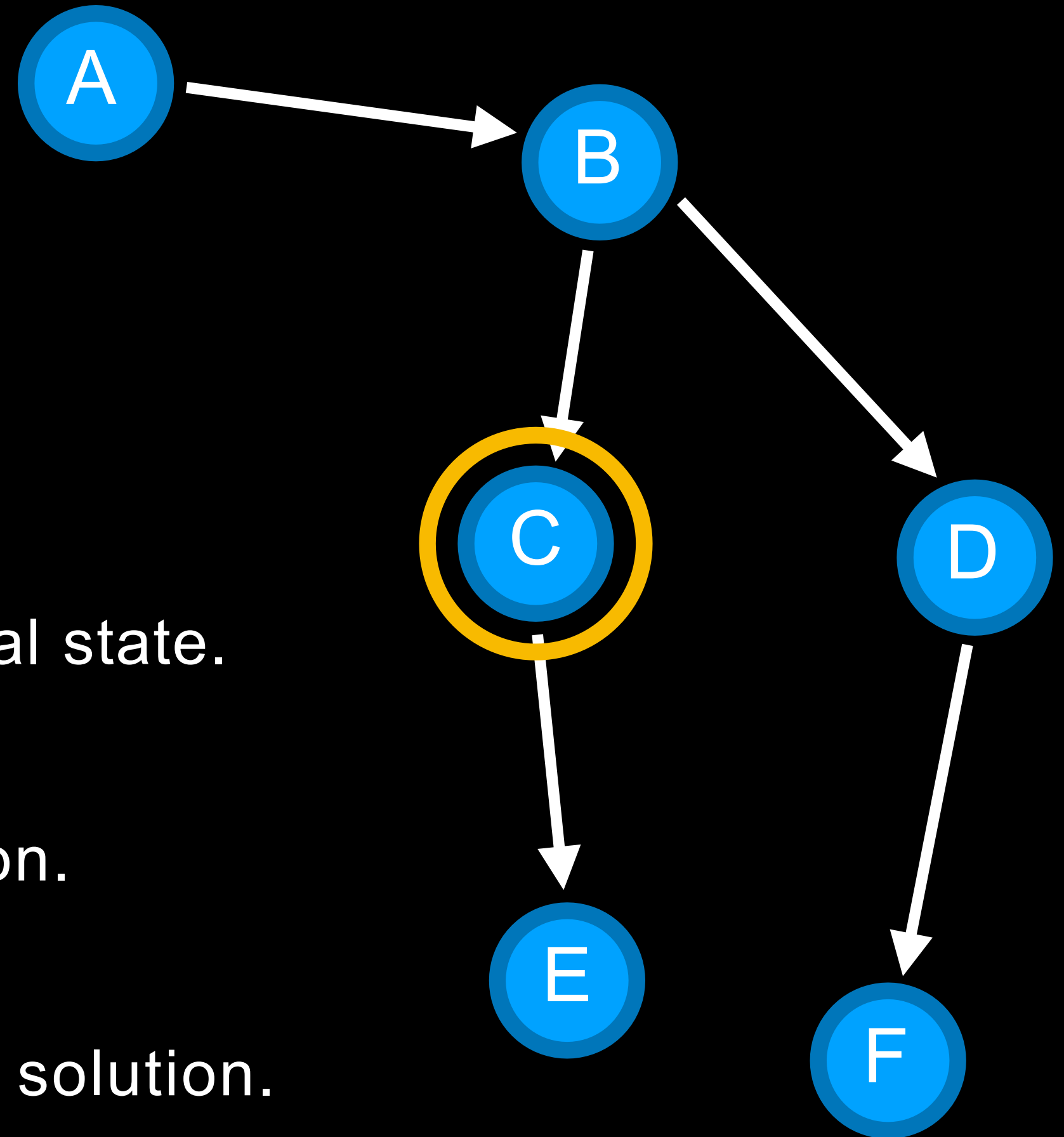


Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

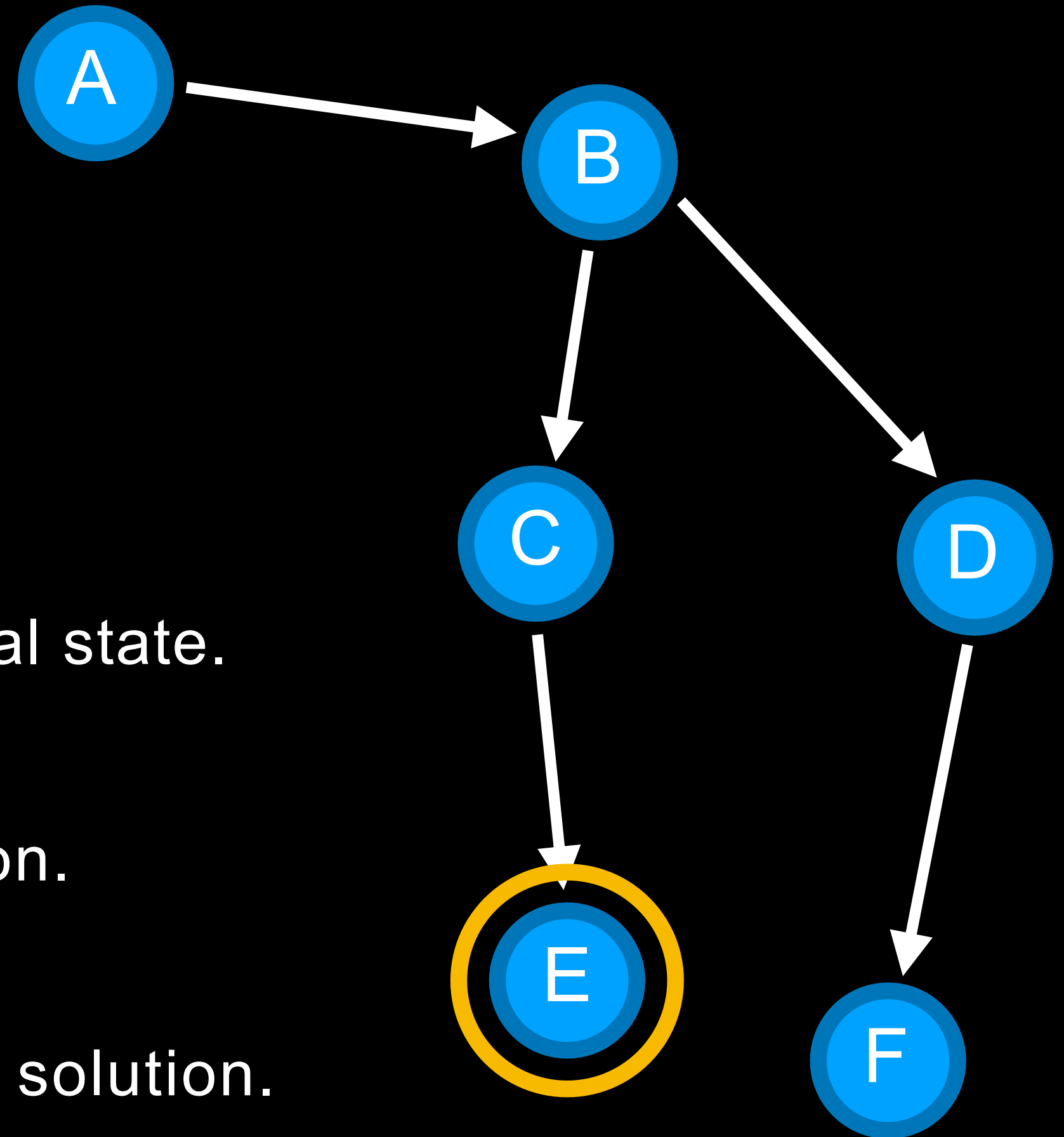


Find a path from A to E.

Frontier



- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.

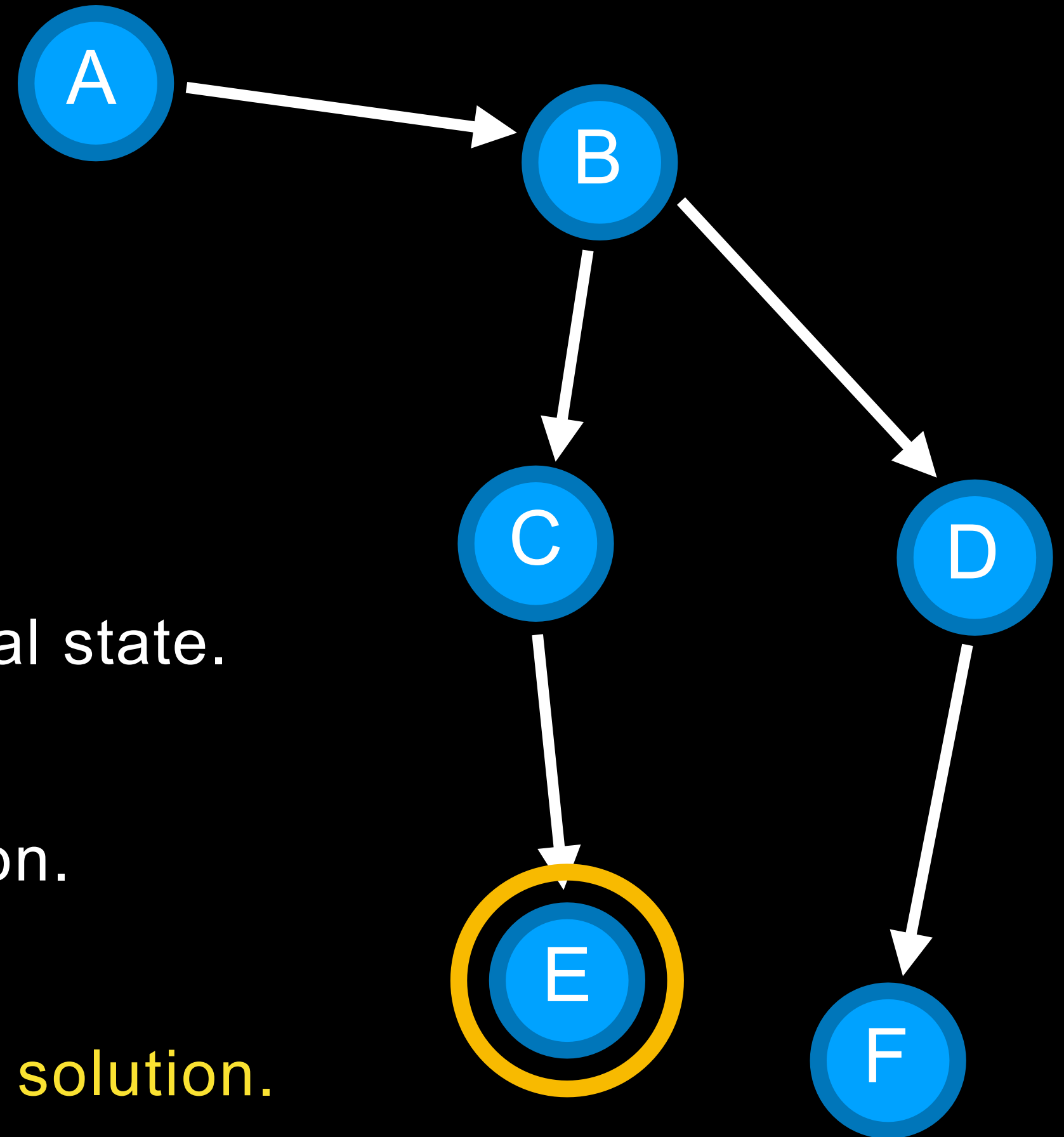


Find a path from A to E.

Frontier



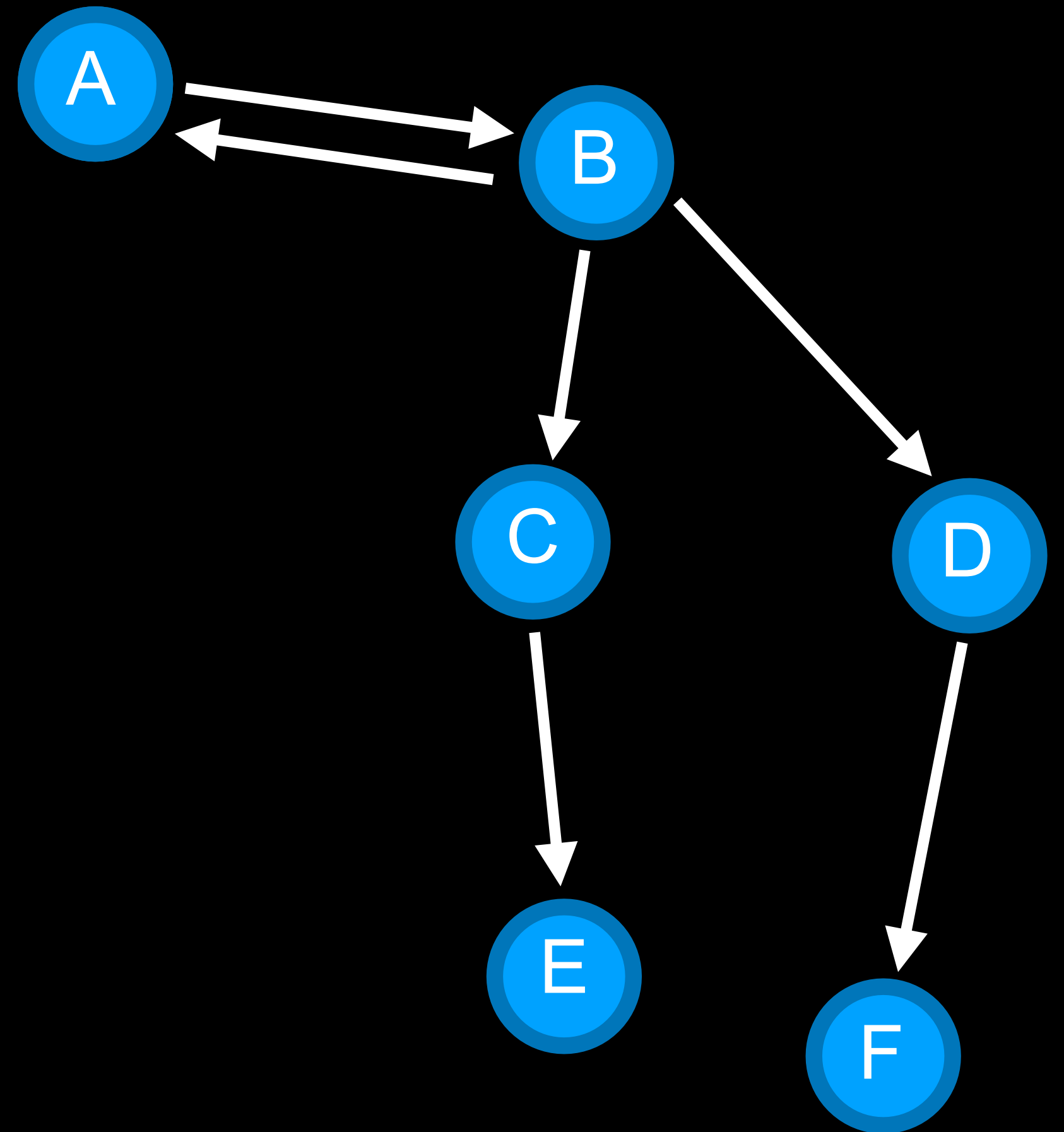
- Start with a **frontier** that contains the initial state.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - **Expand** node, add resulting nodes to the frontier.



What could go wrong?

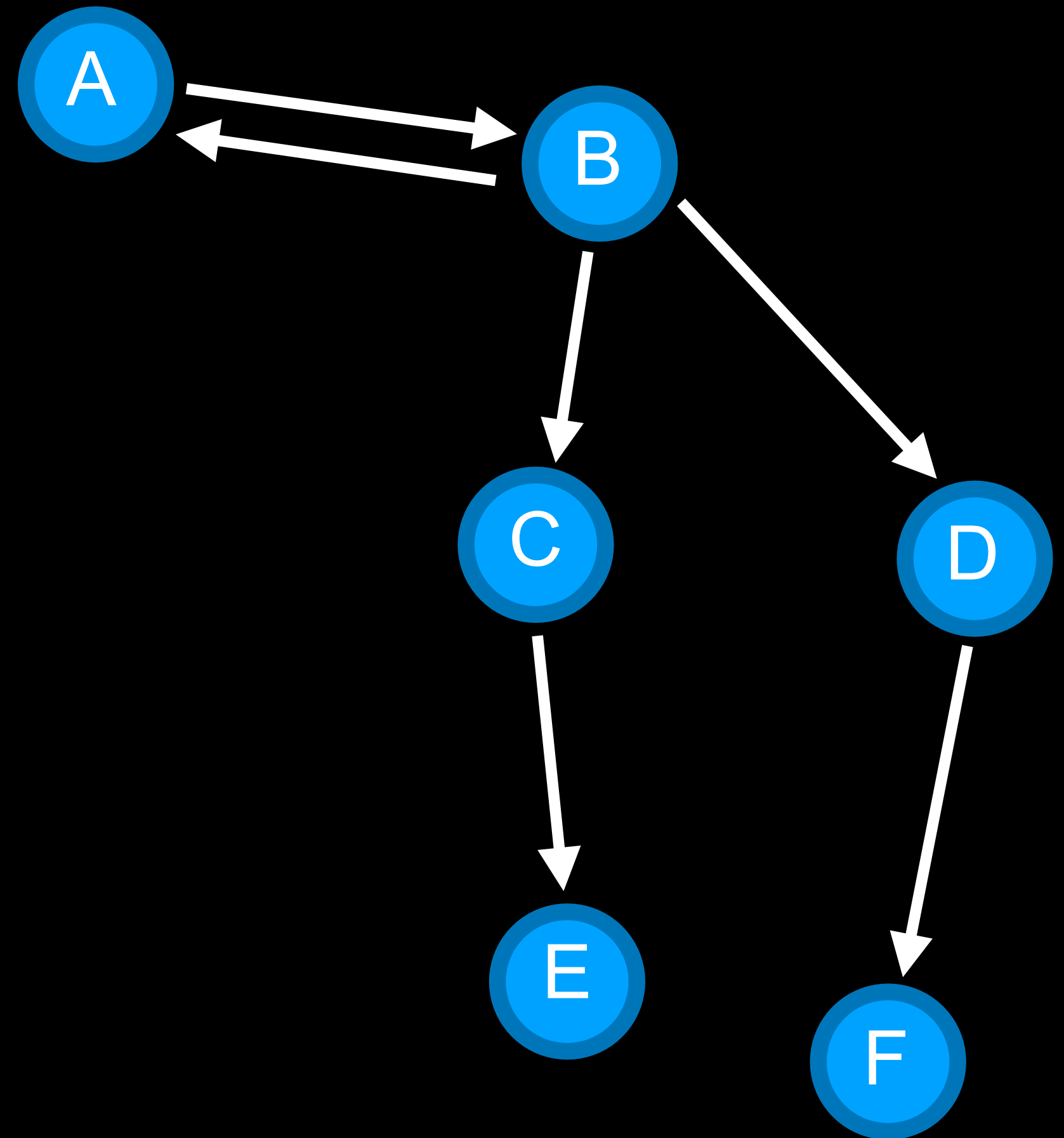
Find a path from A to E.

Frontier



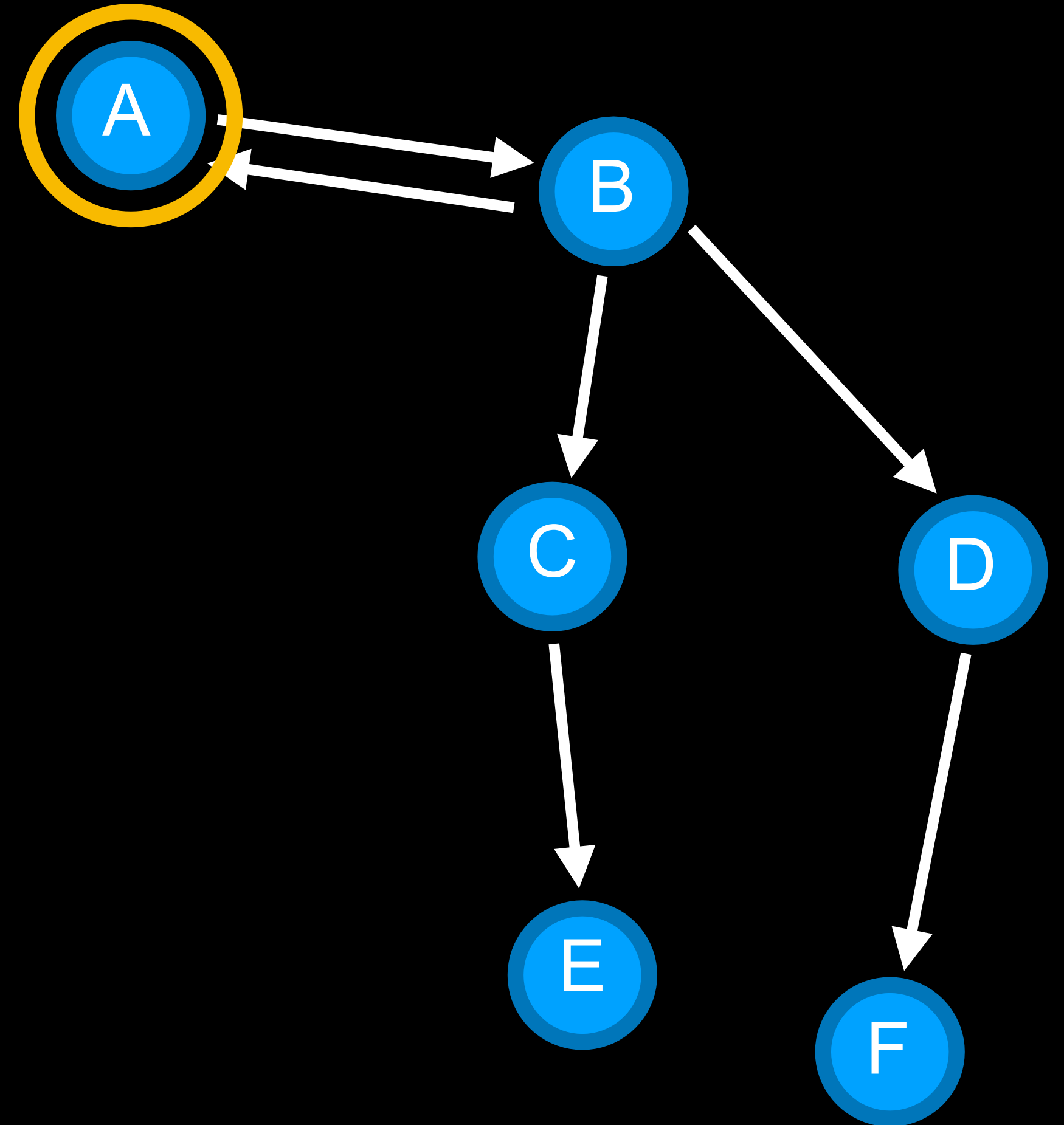
Find a path from A to E.

Frontier



Find a path from A to E.

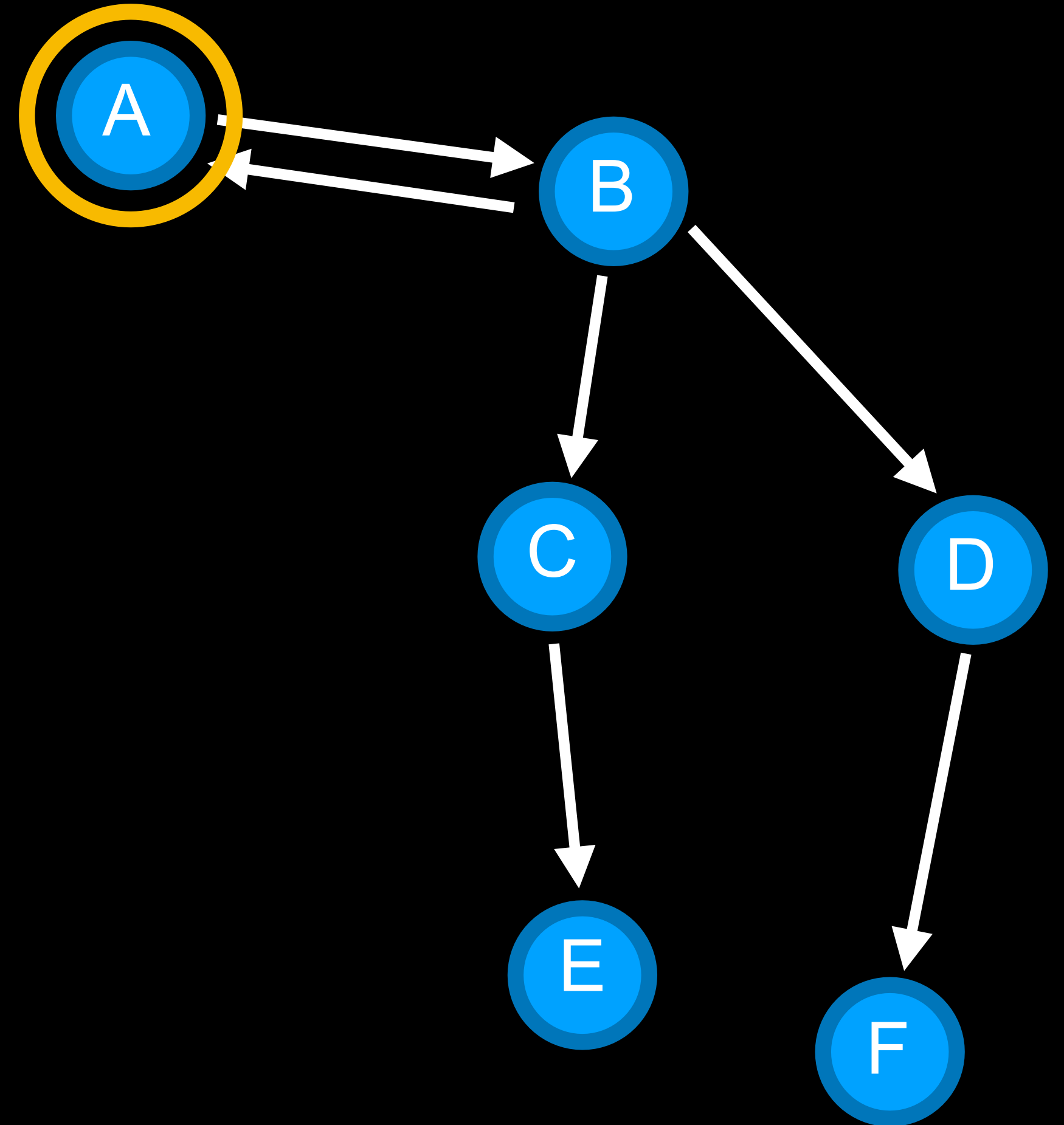
Frontier



Find a path from A to E.

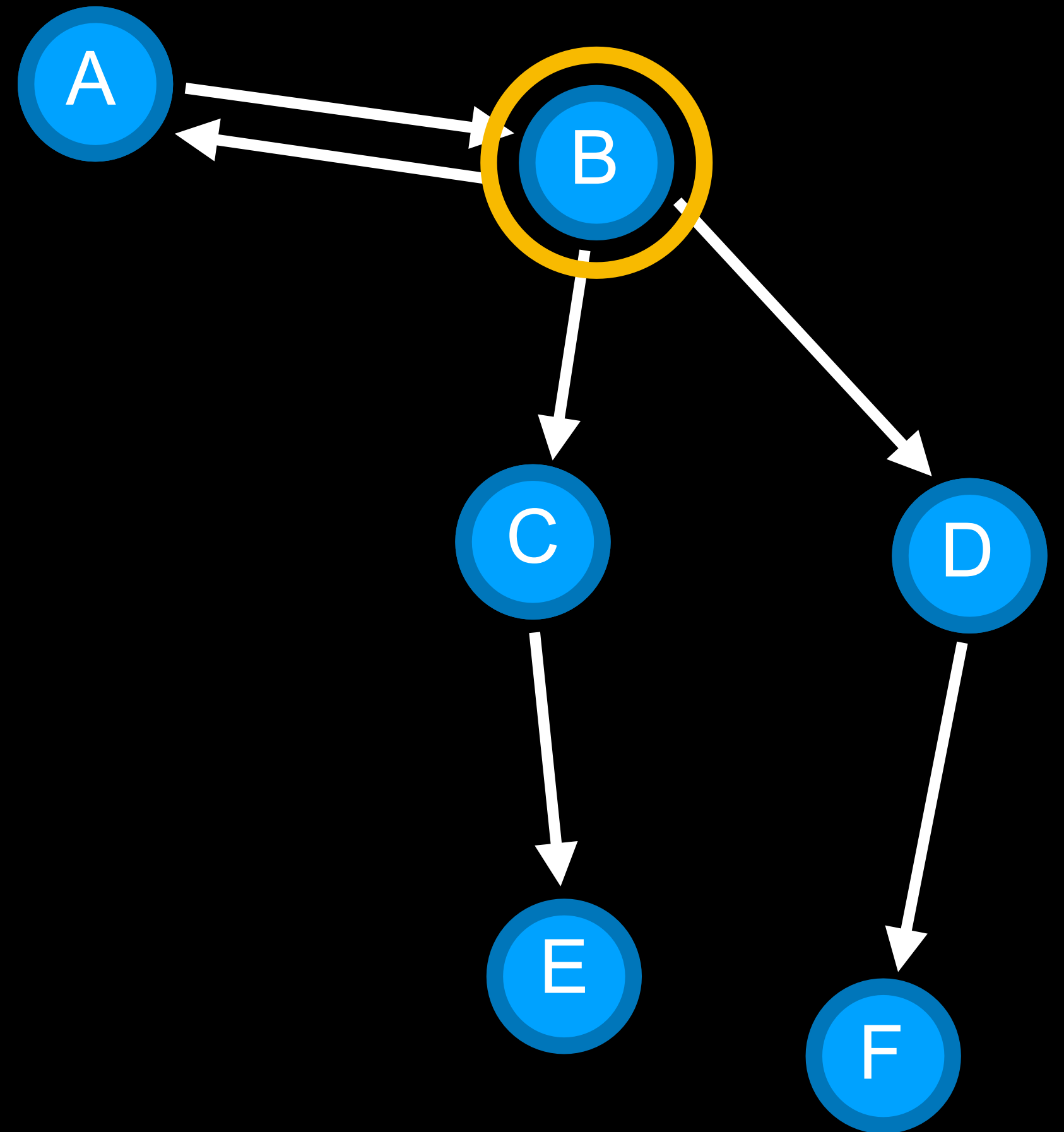
Frontier

B



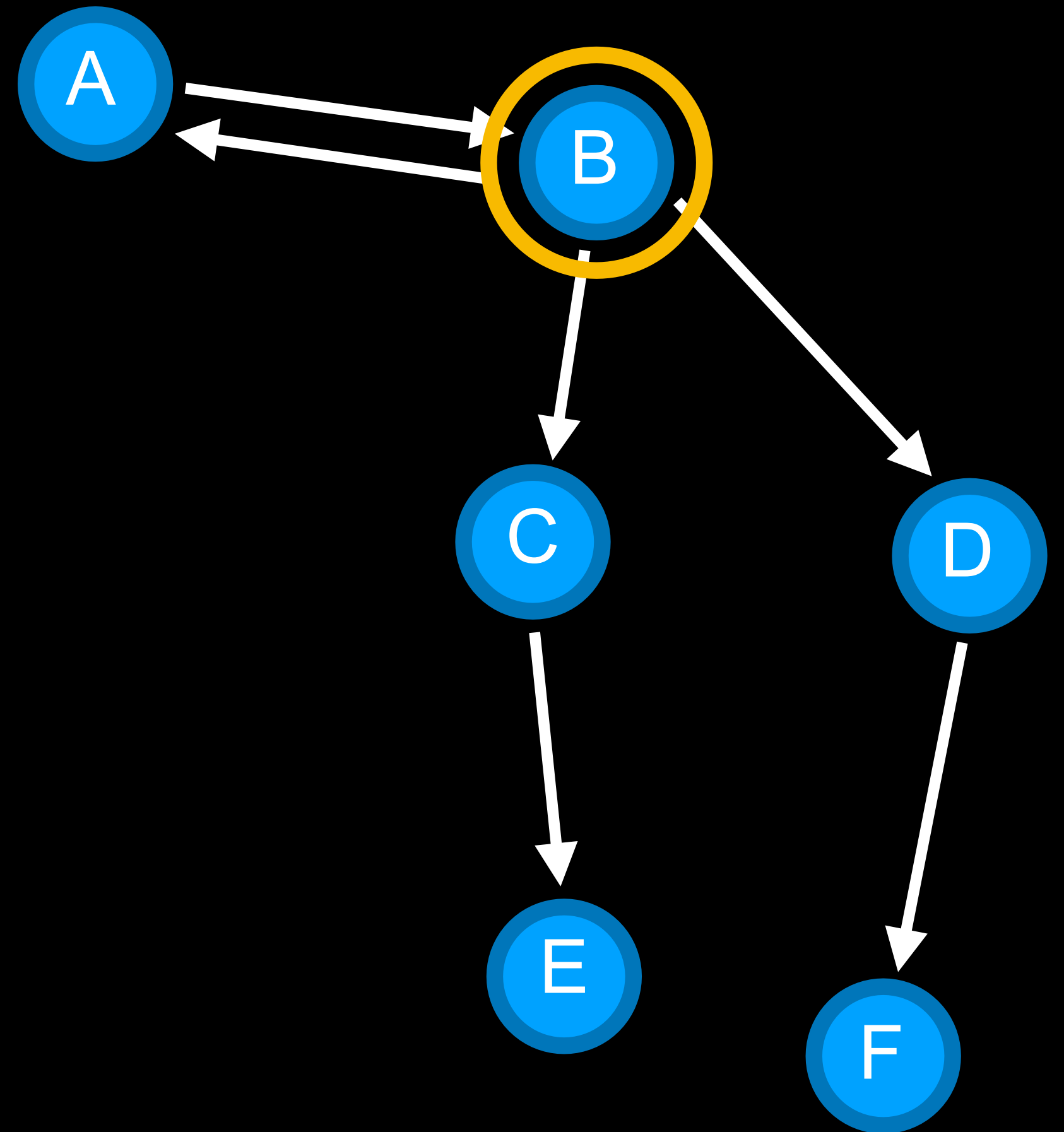
Find a path from A to E.

Frontier



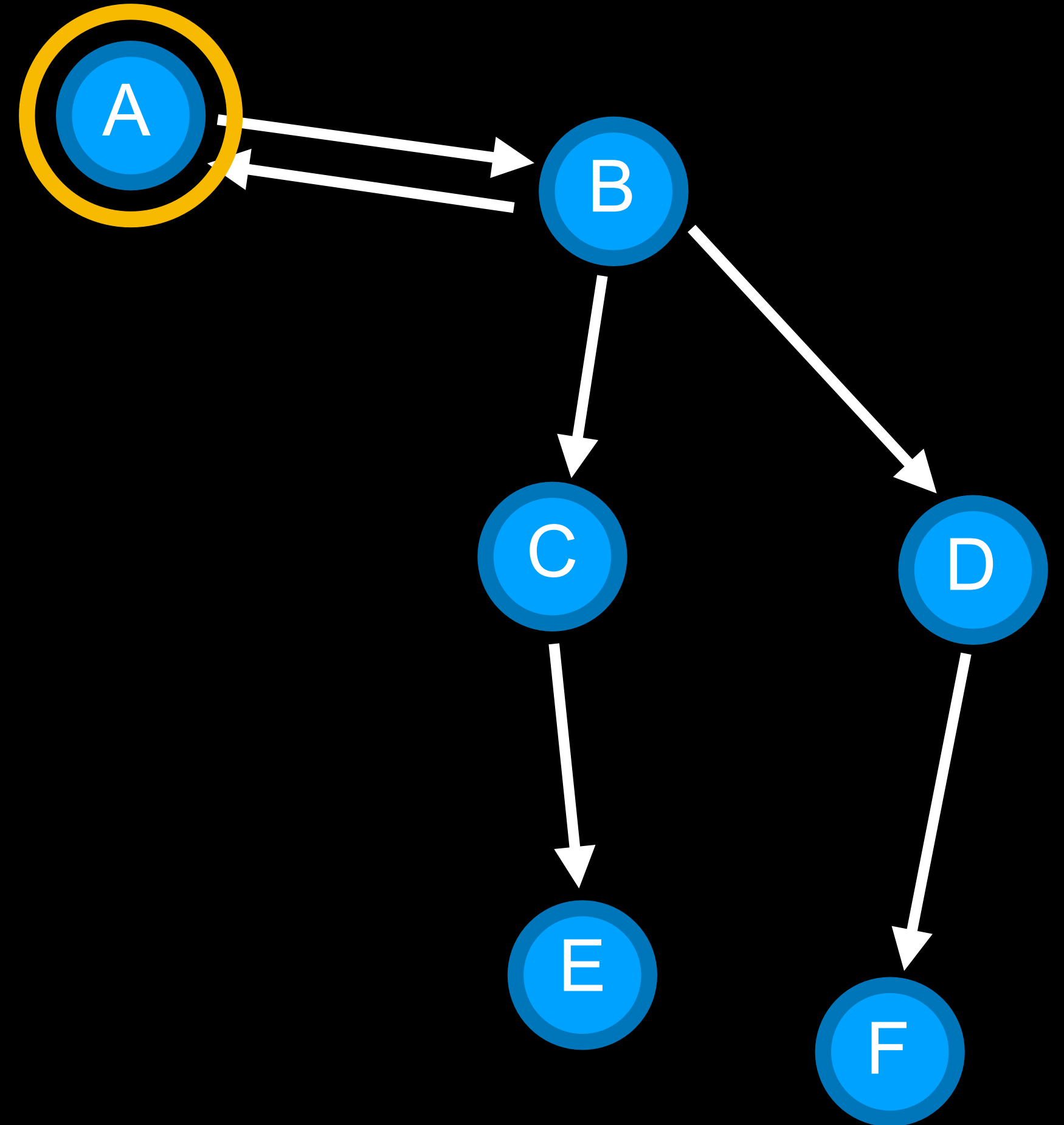
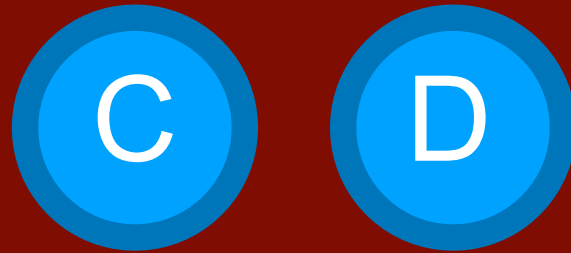
Find a path from A to E.

Frontier



Find a path from A to E.

Frontier



Revised Approach

- Start with a **frontier** that contains the initial state.
- Start with an empty **explored set**.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - Add the node to the explored set.
 - **Expand** node, add resulting nodes to the frontier if they aren't already in the frontier or the explored set.

Revised Approach

- Start with a **frontier** that contains the initial state.
- Start with an empty **explored set**.
- Repeat:
 - If the frontier is empty, then no solution.
 - Remove a node from the frontier.
 - If node contains goal state, return the solution.
 - Add the node to the explored set.
 - **Expand** node, add resulting nodes to the frontier if they aren't already in the frontier or the explored set.

stack

last-in first-out data type

Find a path from A to E.

Frontier

Explored Set

